AI LAB PROGRAMS

Python Source Code Collection

Includes 10 programs covering search algorithms and classic Artificial Intelligence problems.

Programs

1. 8-Puzzle Problem Using A* Search
2. 8-Queens Problem
3. Water Jug Problem Using BFS
4. Cryptarithmetic Problem: SEND + MORE = MONEY
5. Missionaries and Cannibals Problem
6. Vacuum Cleaner Problem
7. Breadth-First Search (BFS)
8. Depth-First Search (DFS)
9. Travelling Salesman Problem (TSP)
10. A* Search Algorithm

1. 8-Puzzle Problem Using A* Search

```
from heapq import heappush, heappop

# Initial and goal states
start = (1, 2, 3,
         4, 0, 6,
         7, 5, 8)

goal = (1, 2, 3,
        4, 5, 6,
        7, 8, 0)


# Manhattan distance heuristic
def heuristic(state):
    distance = 0

    for i, tile in enumerate(state):
        if tile != 0:
            goal_pos = goal.index(tile)
            x1, y1 = divmod(i, 3)
            x2, y2 = divmod(goal_pos, 3)
            distance += abs(x1 - x2) + abs(y1 - y2)

    return distance


# A* search algorithm
def solve(start):
    pq = []
    heappush(pq, (heuristic(start), 0, start, []))
    visited = set()

    while pq:
        f, g, state, path = heappop(pq)

        if state in visited:
            continue

        visited.add(state)
        path = path + [state]

        if state == goal:
            return path

        zero = state.index(0)
        row, col = divmod(zero, 3)

        moves = []
        if row > 0:
            moves.append(zero - 3)
        if row < 2:
            moves.append(zero + 3)
        if col > 0:
            moves.append(zero - 1)
        if col < 2:
            moves.append(zero + 1)

        for pos in moves:
            new_state = list(state)
            new_state[zero], new_state[pos] = (
                new_state[pos], new_state[zero]
            )
            new_state = tuple(new_state)

            if new_state not in visited:
                heappush(
                    pq,
                    (g + 1 + heuristic(new_state),
                     g + 1, new_state, path)
                )

    return None


# Print solution
solution = solve(start)
```

```
if solution:
    print("Solution found!")
    print("Number of moves:", len(solution) - 1)

    for step, state in enumerate(solution):
        print("\nStep", step)
        for i in range(0, 9, 3):
            print(state[i:i + 3])
else:
    print("No solution found.")
```

2. 8-Queens Problem

```
N = 8

def print_solution(board):
    for row in board:
        print(" ".join("Q" if cell else "." for cell in row))

def is_safe(board, row, col):
    # Check the same column
    for i in range(row):
        if board[i][col] == 1:
            return False

    # Check upper-left diagonal
    i, j = row - 1, col - 1
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    # Check upper-right diagonal
    i, j = row - 1, col + 1
    while i >= 0 and j < N:
        if board[i][j] == 1:
            return False
        i -= 1
        j += 1

    return True

def solve(board, row):
    if row == N:
        return True

    for col in range(N):
        if is_safe(board, row, col):
            board[row][col] = 1

            if solve(board, row + 1):
                return True

            # Backtrack
            board[row][col] = 0

    return False

board = [[0 for _ in range(N)] for _ in range(N)]

if solve(board, 0):
    print("Solution for 8 Queen Problem:")
    print_solution(board)
else:
    print("No solution exists")
```

3. Water Jug Problem Using BFS

```
from collections import deque

def water_jug(capacity1, capacity2, target):
    visited = set()
    queue = deque([(0, 0, [])])

    while queue:
        jug1, jug2, path = queue.popleft()

        if (jug1, jug2) in visited:
            continue

        visited.add((jug1, jug2))
        path = path + [(jug1, jug2)]

        if jug1 == target or jug2 == target:
            print("Solution found:")
            for step in path:
                print(step)
            return

        states = [
            (capacity1, jug2),  # Fill jug 1
            (jug1, capacity2),  # Fill jug 2
            (0, jug2),          # Empty jug 1
            (jug1, 0),          # Empty jug 2
        ]

        # Pour jug 1 into jug 2
        pour = min(jug1, capacity2 - jug2)
        states.append((jug1 - pour, jug2 + pour))

        # Pour jug 2 into jug 1
        pour = min(jug2, capacity1 - jug1)
        states.append((jug1 + pour, jug2 - pour))

        for state in states:
            if state not in visited:
                queue.append((state[0], state[1], path))

    print("No solution exists.")

water_jug(4, 3, 2)
```

4. Cryptarithmetic Problem: SEND + MORE = MONEY

```
from itertools import permutations

def solve_cryptarithmetic():
    letters = 'SENDMORY'

    for p in permutations(range(10), len(letters)):
        d = dict(zip(letters, p))

        # First letters cannot be zero
        if d['S'] == 0 or d['M'] == 0:
            continue

        SEND = (d['S'] * 1000 + d['E'] * 100 +
                d['N'] * 10 + d['D'])

        MORE = (d['M'] * 1000 + d['O'] * 100 +
                d['R'] * 10 + d['E'])

        MONEY = (d['M'] * 10000 + d['O'] * 1000 +
                 d['N'] * 100 + d['E'] * 10 + d['Y'])

        if SEND + MORE == MONEY:
            print("Solution found:")
            print("SEND =", SEND)
            print("MORE =", MORE)
            print("MONEY =", MONEY)
            print("Letter assignments:", d)
            return

solve_cryptarithmetic()
```

5. Missionaries and Cannibals Problem

```
from collections import deque

def solve():
    # State: (missionaries_left, cannibals_left, boat_position)
    # boat_position: 0 = left bank, 1 = right bank

    start = (3, 3, 0)
    goal = (0, 0, 1)

    moves = [
        (1, 0),  # One missionary
        (2, 0),  # Two missionaries
        (0, 1),  # One cannibal
        (0, 2),  # Two cannibals
        (1, 1)   # One missionary and one cannibal
    ]

    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        state, path = queue.popleft()
        m, c, boat = state

        if state == goal:
            print("Solution found:")
            for s in path:
                print(s)
            return

        for dm, dc in moves:
            if boat == 0:
                new_state = (m - dm, c - dc, 1)
            else:
                new_state = (m + dm, c + dc, 0)

            nm, nc, nb = new_state

            # Check population limits
            if not (0 <= nm <= 3 and 0 <= nc <= 3):
                continue

            # Check safety on both banks
            if nm > 0 and nc > nm:
                continue

            mr, cr = 3 - nm, 3 - nc
            if mr > 0 and cr > mr:
                continue

            # Check that the boat carries at least one person
            if dm + dc == 0:
                continue

            if new_state not in visited:
                visited.add(new_state)
                queue.append((new_state, path + [new_state]))

    print("No solution exists.")

solve()
```

6. Vacuum Cleaner Problem

```
def vacuum_cleaner():
    # 0 = Clean, 1 = Dirty
    room = {
        'A': 1,
        'B': 1
    }

    position = 'A'

    print("Initial state:", room)

    while 1 in room.values():
        if room[position] == 1:
            print("Room", position, "is Dirty. Cleaning...")
            room[position] = 0
        else:
            print("Room", position, "is Clean.")

        # Move to the other room
        if position == 'A':
            position = 'B'
        else:
            position = 'A'

    print("All rooms are clean!")
    print("Final state:", room)

vacuum_cleaner()
```

7. Breadth-First Search (BFS)

```
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()

        if node not in visited:
            print(node, end=" ")
            visited.add(node)

            for neighbor in graph[node]:
                if neighbor not in visited:
                    queue.append(neighbor)

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

print("Breadth First Search Traversal:")
bfs(graph, 'A')
```

8. Depth-First Search (DFS)

```
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in graph[node]:
            dfs(graph, neighbor, visited)

# Graph representation
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

print("Depth First Search Traversal:")
dfs(graph, 'A')
```

9. Travelling Salesman Problem (TSP)

```
from itertools import permutations

def tsp(graph, start):
    cities = list(graph.keys())
    cities.remove(start)

    min_cost = float('inf')
    best_path = None

    for path in permutations(cities):
        route = [start] + list(path) + [start]
        cost = 0

        for i in range(len(route) - 1):
            cost += graph[route[i]][route[i + 1]]

        if cost < min_cost:
            min_cost = cost
            best_path = route

    print("Shortest Path:", " -> ".join(best_path))
    print("Minimum Cost:", min_cost)


# Distance between cities
graph = {
    'A': {'A': 0, 'B': 10, 'C': 15, 'D': 20},
    'B': {'A': 10, 'B': 0, 'C': 35, 'D': 25},
    'C': {'A': 15, 'B': 35, 'C': 0, 'D': 30},
    'D': {'A': 20, 'B': 25, 'C': 30, 'D': 0}
}

tsp(graph, 'A')
```

10. A* Search Algorithm

```
def astar(graph, heuristic, start, goal):
    open_list = [(heuristic[start], 0, start, [start])]
    visited = set()

    while open_list:
        open_list.sort()
        f, g, node, path = open_list.pop(0)

        if node == goal:
            print("Shortest path:", " -> ".join(path))
            print("Total cost:", g)
            return

        if node in visited:
            continue

        visited.add(node)

        for neighbor, cost in graph[node]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]

                open_list.append(
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    print("Path not found")


# Graph with edge costs
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('F', 3)],
    'D': [('G', 5)],
    'E': [('G', 2)],
    'F': [('G', 2)],
    'G': []
}

# Heuristic values to the goal G
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 4,
    'E': 2,
    'F': 1,
    'G': 0
}

astar(graph, heuristic, 'A', 'G')
```